

Urban Flighter: An Honest Urban-Air Simulator with Hidden CFD-lite Flow and Quadratic Air-Relative Drag

Jaewon Jang

Inha University

August 2026

Abstract

Urban air routing needs a simulator in which buildings, local wind, drag, sensing, and energy interact, without leaking privileged flow fields into a learning agent. Urban Flighter is an open-source urban drone simulator that couples real OpenStreetMap geometry and forecast-model inlet wind with a browser cockpit and a headless research environment. The contribution is not a new Navier–Stokes solver. It is an explicit contract: cheap urban-flow approximations and quadratic air-relative drag may drive dynamics, energy, and metrics, while a policy may observe only mission context, the known inlet, onboard relative-air estimates, and deterministic geometry LiDAR. We document the flyable 2D / 3D Lite loop, the potential-flow CFD-lite field with wall damping and empirical wake correction, a momentum-theory induced plus quadratic parasite power model, and UrbanFlow Gym, which is RL-ready but not trained. Two checkable studies are reported. A still-air / headwind / tailwind integration of the shared drag core shows stick-off drift toward local wind and a higher energy cost in headwind than tailwind under the same thrust. An offline Gym fixture shows that unconstrained goal-seeking collides, geometry-safe A* succeeds, and an inlet-aware heuristic is not automatically energy-optimal.

1 Introduction

Small uncrewed aircraft in cities feel two things that game physics usually erases: buildings that must not be hit, and air that is not still. A tailwind lowers relative airspeed; a headwind raises it; a wake behind a slab can do both in one block. Those effects change both the trajectory a vehicle can hold and the energy it burns [4].

Existing aerial simulators often optimize photorealism or vehicle dynamics [2, 5]. Standalone CFD packages optimize field accuracy and are not interactive flight products. Urban Flighter sits between them. It loads real OpenStreetMap (OSM) footprints when a city is requested [3], computes a cheap hidden flow field, flies a metre-scale quadrotor under quadratic air-relative drag, and exposes a Gymnasium-shaped interface [1] whose actor channel is forbidden from seeing the grid.

This paper is a system / software preprint, not a turbulence-model paper and not a trained-policy paper. Every quantitative number below is produced by committed scripts.

Contributions.

1. An honest full-stack urban-air cockpit: OSM geometry, CFD-lite hidden flow, 120 Hz flyable 2D and 3D motion, and simulator LiDAR maps that are labelled as odometry, not SLAM.

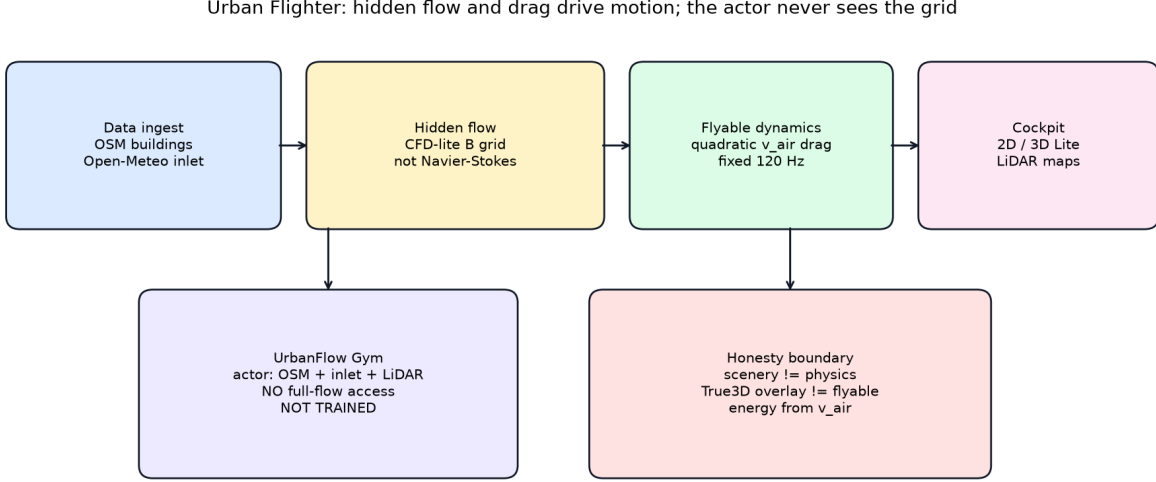


Figure 1: Hidden CFD-lite flow and quadratic drag drive motion and energy. The actor never receives the grid. Scenery and the True 3D $u/v/w$ overlay are display-only.

2. A shared quadratic air-relative drag core used by flyable motion and energy accounting, with momentum-theory induced power. Stick-off equilibrium is local wind, not still air.
3. UrbanFlow Gym: a NumPy environment with leakage guards, live-scenario binding, and three deterministic baselines. Status: NOT TRAINED.
4. Two reproducible studies (drag integration; offline Gym fixture) whose qualitative outcomes are stated in advance and not rewritten after the run.

2 Related work

AirSim provides Unreal-based visual and physical simulation for autonomous vehicles, including a quadrotor, and compares software components with real flights [5]. FlightGoggles emphasizes photorealistic exteroceptive sensing and can keep complex vehicle dynamics outside the renderer [2]. Urban Flighter does the opposite of photogrammetry-first design: the primary object is a research contract over urban wind, drag, and observations, rendered in a browser rather than Unreal or Unity.

Hoffmann et al. showed that even moderate airspeed changes quadrotor thrust and attitude, so hover-only models fail outdoors [4]. We take that lesson at a coarser grain: parasite drag and induced power depend on $\mathbf{v}_{\text{air}} = \mathbf{v}_g - \mathbf{w}$, not on ground speed alone. The model is not blade-element theory and is not a claim to match STARMAC flight data.

OpenStreetMap supplies the building layer [3]. The Gym-style reset/step shape follows the OpenAI Gym interface [1]. Gymnasium / Stable-Baselines3 / Torch remain optional extras; the default UrbanFlow core is NumPy-only.

3 System

Figure 1 is the product split.

Backend. /flow-fields/2d returns OSM polygons, an Open-Meteo (or labelled fallback) inlet, and a streamfunction / potential-flow-ish grid with wall damping and an empirical wake correction (“CFD-lite B”). A successful non-empty response is registered as an immutable snapshot urban-flow.live_scenario.v1.

Frontend. Three public modes share the same flyable horizontal grid:

- 2D: north-up canvas, 120 Hz, swept-circle clearance, 180-ray scan.
- 3D Lite: 0.58 m visual quad, 1.25 m horizontal / 2 m roof research envelope, building and ground returns.
- True 3D Wind: the same flyable city plus a bundled voxel potential-flow $u/v/w$ overlay. Overlay $\neq$ flight dynamics.

Presentation trees, lamps, and road paint never enter collision, LiDAR, Gym, reward, or scenario hashes.

Honesty labels. CFD-lite $\neq$ Navier–Stokes; SIM odometry / no loop closure; UrbanFlow Gym / NOT TRAINED; full flow access = NO.

4 Hidden flow

A 2D slice solves a discrete Laplace problem on a masked grid and recovers a velocity field from the potential or streamfunction. Inlet / far-field velocity is prescribed; building cells are solid. This is cheap and produces acceleration around corners (Fig. 2). It cannot represent viscous separation, vortex shedding, or turbulence. Empirical wall damping and downstream wake patches are therefore added for drone-risk texture and must be named as such.

A voxel potential-flow field can supply u, v, w streamlines for visualization. The flyable aircraft still samples the live horizontal CFD-lite grid. No Navier–Stokes residual is claimed.

5 Quadratic air-relative drag

Let $\mathbf{v}_g$ be ground velocity and $\mathbf{w}$ the hidden local wind. Relative air velocity is

$$\mathbf{v}_{\text{air}} = \mathbf{v}_g - \mathbf{w}. \quad (1)$$

Parasite force and power use the standard quadratic law

$$\mathbf{F}_p = -\frac{1}{2}\rho C_d A \|\mathbf{v}_{\text{air}}\| \mathbf{v}_{\text{air}}, \quad P_p = \frac{1}{2}\rho C_d A \|\mathbf{v}_{\text{air}}\|^3. \quad (2)$$

The flyable integrators apply an implicit step that is unconditionally stable at the 120 Hz cockpit rate and at the larger diagnostic steps used in collision tests:

$$\mathbf{v}_{\text{air}}^+ = \frac{\mathbf{v}_{\text{air}}}{1 + \Delta t(k\|\mathbf{v}_{\text{air}}\| + c)}, \quad k = \frac{\rho C_d A}{2m}. \quad (3)$$

With thrust off, the only equilibrium is $\mathbf{v}_g = \mathbf{w}$. That is the difference from the previous linear ground-velocity damper, which stopped the vehicle in still air even inside a wind field.

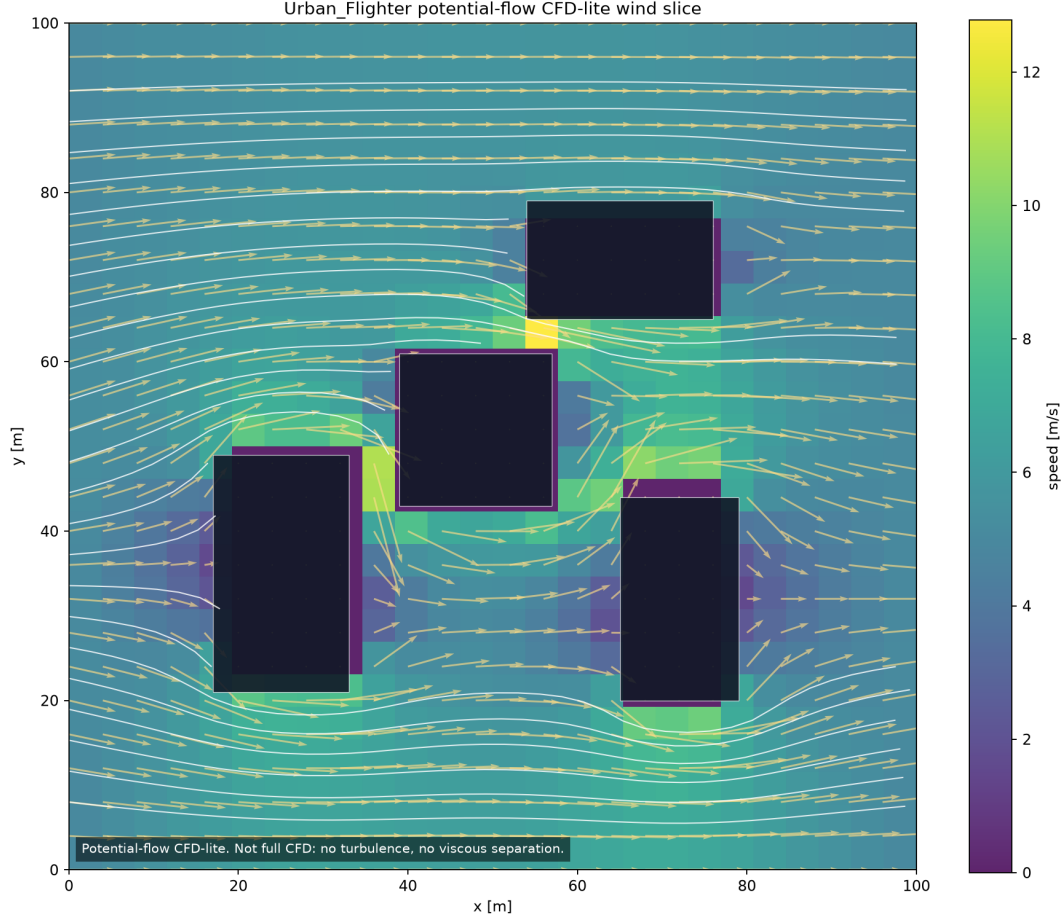


Figure 2: Offline toy-city potential-flow CFD-lite showcase. Five prisms, 5 m/s westerly inlet, 26×26 grid, 348 iterations, residual 9.84×10^{-5} . Mean speed 4.96 m/s, max speed 12.8 m/s. Not Navier–Stokes.

Induced power uses momentum theory, not blade element:

$$v_h = \sqrt{\frac{W}{2\rho A_{\text{disk}}}}, \quad P_i = \frac{\kappa W v_h}{\sqrt{1 + (\|\mathbf{v}_{\text{air}}\|/v_h)^2}}. \quad (4)$$

Total electrical-equivalent power is avionics + sensors + P_p + P_i + climb $W \max(v_z, 0)$. Default constants: $m = 2.5$ kg, $\rho = 1.225$ kg/m³, $C_d = 1.05$, $A = 0.18$ m², four 0.15 m propellers, $\kappa = 1.15$. Then $k = 0.0463$ m⁻¹.

UrbanFlow Gym still uses a linear wind-drag gain by default so the published RL contract does not silently change. The quadratic core is shared by the cockpit, a Python module, and a Unity C# port of the same formulas. The Unity file is not a scene rewrite.

6 Sensing and the actor contract

Frontend LiDAR prototypes use a frozen five-value-per-ray layout $(d_x, d_y, d_z, \hat{r}, h)$: local direction, normalized range, and hit flag. 3D default: 600 Fibonacci-sphere samples, 120 m. 2D default: 180

rays, 180 m. Ground is a real target: downward rays intersect $y = 0$ and take the nearer of ground and buildings.

UrbanFlow Gym uses a 33-scalar actor observation: own motion, goal context, fixed-angle geometry LiDAR, known inlet, previous action, and an onboard relative-air estimate. Automated leakage guards reject full grids, exact wakes, future samples, and provider objects. A privileged critic helper exists and is never returned by reset/step.

7 UrbanFlow Gym

The core is NumPy-only. Action is a bounded 2D local-guidance command, not motor RPM. Three deterministic baselines share only public mission context:

1. Direct goal (no obstacle planner).
2. Geometry-safe A^* + waypoint tracking.
3. Inlet-aware A^* with known-inlet feed-forward. Still not privileged-grid access.

Live evaluation binds to the exact UI-registered OSM scenario. Section 9 uses the synthetic rectangular fixture so the committed study needs no network.

8 Study A: drag essence

Command:

`PYTHONPATH=backend python scripts/render_paper_figures.py`

The same core is also exercised by `scripts/run_drag_essence_demo.py`.

Still-air power split. Figure 3 integrates the analytic power model over $|\mathbf{v}_{\text{air}}| \in [0, 18]$ m/s. Hover total power is 362 W. The minimum is 350 W at 5.2 m/s. At 11 m/s the total is 427 W: induced has fallen, parasite has risen. This is the expected U-shape of a coarse momentum-plus-parasite model, not a fitted motor map.

Same thrust, three winds. A constant 8 m/s^2 body thrust is integrated for 8 s at $\Delta t = 1/120$ s (Fig. 4, left). Final ground speeds: tailwind $+6 \text{ m/s} \rightarrow 16.4 \text{ m/s}$; still air $\rightarrow 10.4 \text{ m/s}$; headwind $-6 \text{ m/s} \rightarrow 4.4 \text{ m/s}$. Integrated energy: 3116 J, 3151 J, 3203 J. Headwind is slower and costlier. The energy gap is modest because the run is short and induced still dominates at these speeds; the speed gap is not.

Stick-off. Zero thrust in an 8 m/s wind reaches 7.28 m/s ground speed in 6 s (Fig. 4, right). The vehicle is drifting with the air, not parking in the world frame.

9 Study B: Gym fixture

Command (no network):

`PYTHONPATH=backend python scripts/run_oss_showcase.py`

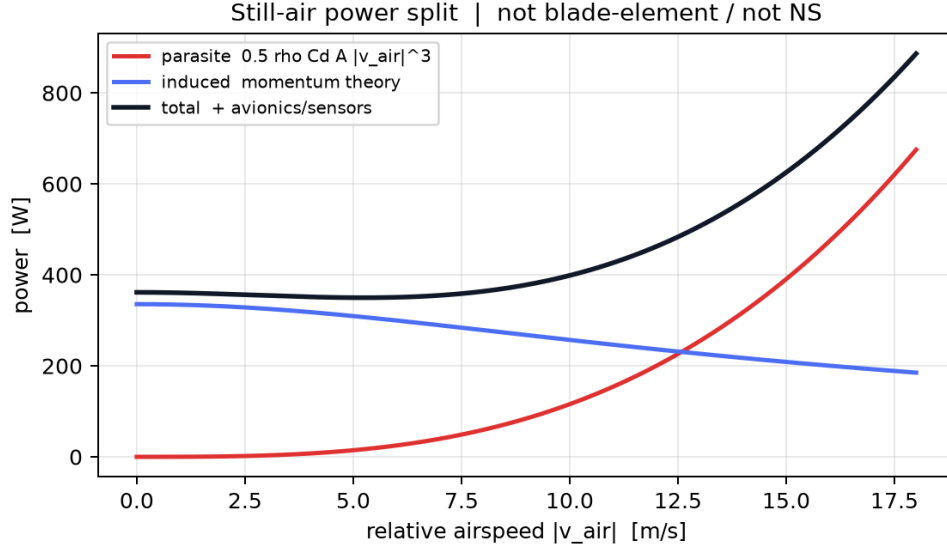


Figure 3: Still-air power versus relative airspeed. Induced falls, parasite rises. Not blade-element theory.

Table 1: UrbanFlow Gym synthetic-fixture evaluation. Inlet-aware is not energy-better on this set; the regression is reported, not hidden.

Baseline	Success	Mean path (m)	Rel-air energy	Collision eps.
Direct goal	0/3	23.5	180	3/3
Shortest-path A*	3/3	142.1	1236	0/3
Inlet-aware A*	3/3	141.9	1303	0/3

Seeds {10007,10009,10037}, horizon 240 steps, $\Delta t = 0.25$ s. Policy status: NOT TRAINED. Privileged flow access: false. Hidden dynamics: synthetic potential-flow-ish wake proxy, not Navier–Stokes. Evaluation id: 24ec026f3352e4f9.

The showcase is a contract test. A successful public demo must show (i) collisions when geometry is ignored, (ii) success when it is not, (iii) no privileged-grid leakage, and (iv) honest labels.

10 Limitations

- CFD-lite and voxel potential flow are not LES/RANS/DNS.
- True 3D Wind is a visualization overlay.
- Flyable motion is a damped point-mass with quadratic air drag, not rotor flapping or blade-element moments [4].
- LiDAR and rolling maps are simulator rays plus odometry, not hardware SLAM.
- No PPO/SAC training is shipped.
- Live OSM/weather paths need network; the committed Gym showcase does not.

Quadratic air-relative drag in the flyable loop

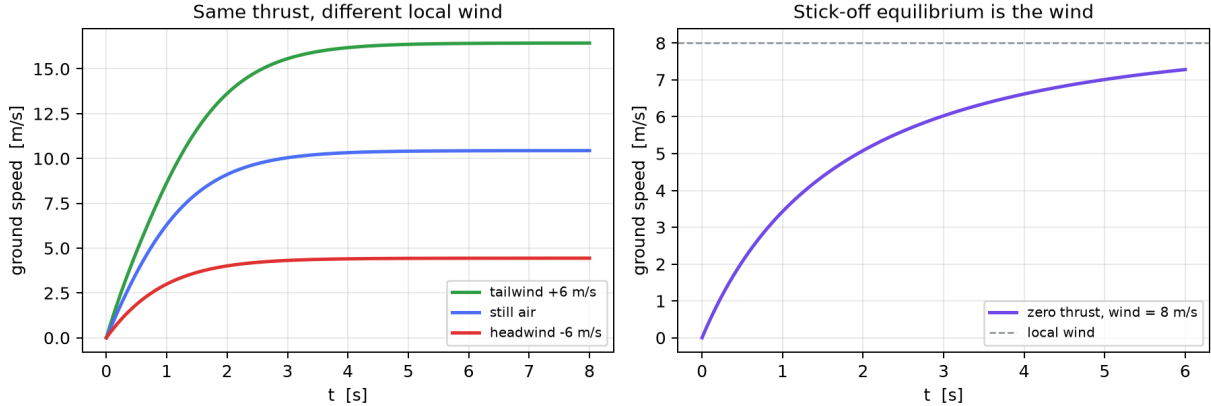


Figure 4: Left: identical thrust in tailwind, still air, and headwind. Right: thrust off; ground speed approaches the local wind.

- Presentation geometry never enters physics.
- Gym still uses linear wind drag by default.

11 Reproducibility

- Code: https://github.com/VortexyAether/Urban_Flighter
- License: MIT.
- Drag figures: Section 8.
- Gym figures and Table 1: Section 9.
- Paper figures: paper/figures/, metrics in paper/figures/paper_figure_metrics.json.

12 Conclusion

Urban Flighter is an open urban-air research cockpit: real cities when requested, cheap hidden flow, quadratic air-relative drag in the flyable loop, and a deliberately narrow actor contract. The public numbers are intentionally small and checkable. That is the point.

References

- [1] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016.
- [2] Winter Guerra, Ezra Tal, Varun Murali, Gilhyun Ryou, and Sertac Karaman. FlightGoggles: A modular framework for photorealistic camera, exteroceptive sensor, and dynamics simulation. In IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2019. arXiv:1905.11377.

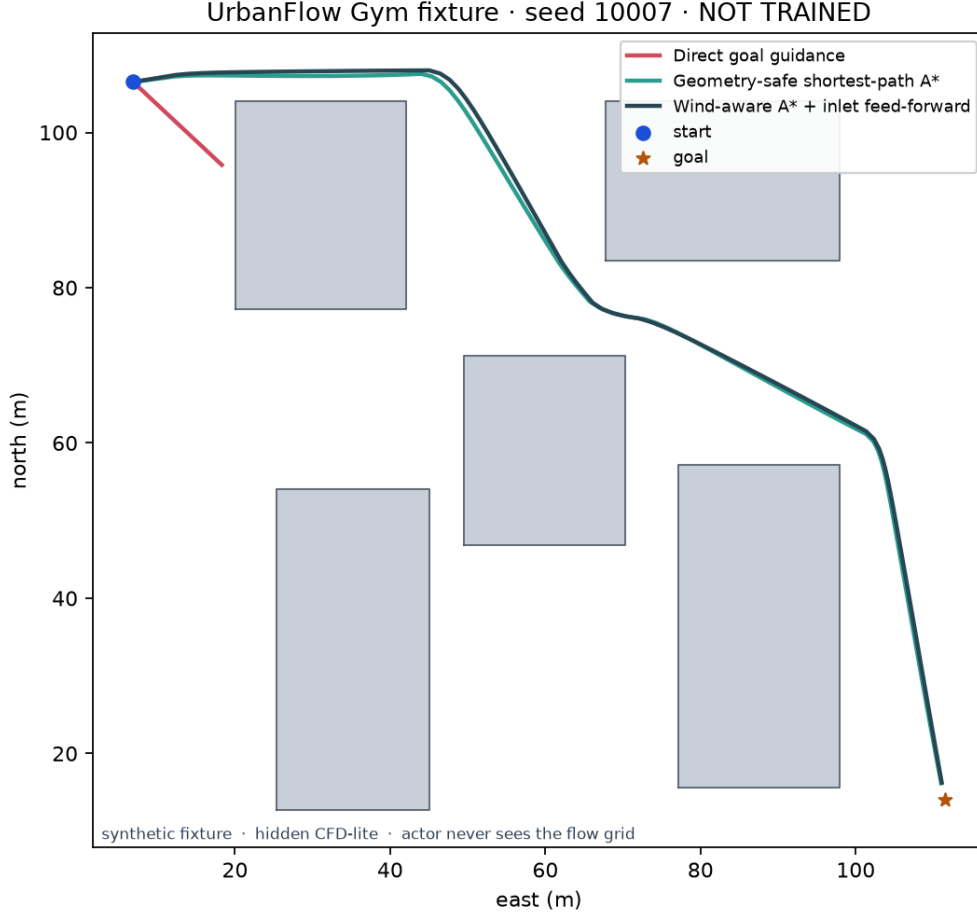


Figure 5: Seed 10007. Direct goal collides. Both planners reach the goal. Actor observation dimension is 33.

- [3] Mordechai Haklay and Patrick Weber. OpenStreetMap: User-generated street maps. *IEEE Pervasive Computing*, 7(4):12–18, 2008.
- [4] Gabriel M. Hoffmann, Haomiao Huang, Steven L. Waslander, and Claire J. Tomlin. Quadrotor helicopter flight dynamics and control: Theory and experiment. In *AIAA Guidance, Navigation, and Control Conference and Exhibit*, Hilton Head, SC, 2007.
- [5] Shital Shah, Debadeepta Dey, Chris Lovett, and Ashish Kapoor. AirSim: High-fidelity visual and physical simulation for autonomous vehicles. In *Field and Service Robotics*, 2018. arXiv:1705.05065.

A Software map

backend/ FastAPI and CFD-lite;
 backend/urban_fighter_physics/ shared drag;
 backend/urbanflow_gym/ headless env;
 frontend/src/simulation/quadraticAirDrag.ts cockpit twin;

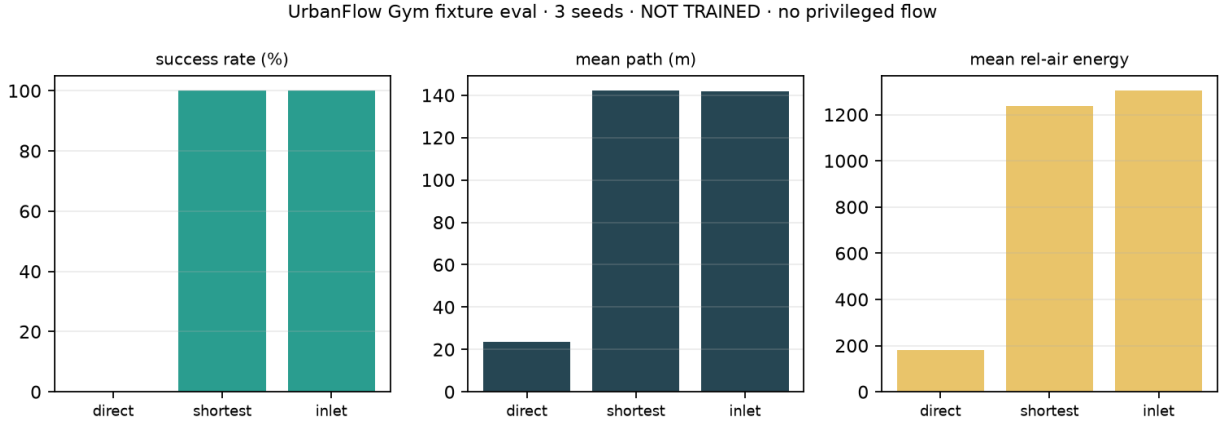


Figure 6: Aggregate fixture metrics. Geometry, not inlet heuristics, is what separates success from collision here.

unity/UrbanFlighterDrag/ formula port only;
scripts/render_paper_figures.py and scripts/run_oss_showcase.py this paper.